Rebuilding the MLA Data Tools from Scratch: a 16-Step Prompt Guide

This document is a sequence of prompts you can paste into Claude Code (or another AI coding assistant). Followed in order, they reproduce a project with the same structure and features as `moon-mla-app`: ten MLA Statistics API fetchers, a weekly merge script, a Streamlit web app, a unified `mla` command line, and a set of Claude Code skills.

How to use this guide

- Open Claude Code in an empty folder and start at Step 1, one prompt per step.
- After each step, run the checks listed under **Verify**. Commit when they pass, then move to the next step.
- Replace `YOUR_EMAIL@example.com` in the prompts with your own address. MLA's terms of use ask that requests carry a contact.
- Prerequisites: Python 3.9 or newer and network access to `https://api-mlastatistics.mla.com.au`.

Step 3 is the heart of the project: the fetcher it produces is the template that the following seven steps copy. Step 15 creates the skills that let you reach MLA data from any later conversation.

Step 1 — Project skeleton

Goal: folder layout, licence, dependency file, and behavioural guidelines for the assistant.

```
Create a new Python project called `mla-stats-fetcher` for downloading Australian
livestock data from the MLA Statistics API (https://api-mlastatistics.mla.com.au).

Set up this folder layout (create empty folders with a .gitkeep where needed):

  fetchers/          one standalone CLI script per API endpoint (add __init__.py)
  analysis/          post-processing scripts (add __init__.py)
  data/raw/          CSV output of the fetchers
  data/processed/    derived tables
  assets/            logo and static files for the web app
  docs/              additional documentation

At the root create:
  - README.md with a one-paragraph description and a "Project Structure" section
    showing the tree above. Say that all commands are run from the repository root.
  - requirements.txt containing `streamlit>=1.32` and `pandas>=2.0`
    (the fetchers themselves must stay standard-library only; these are for
    the web app and analysis scripts added later).
  - .gitignore for Python: __pycache__/, *.pyc, .venv/, venv/, *.egg-info/,
    build/, dist/, .DS_Store
  - LICENSE (MIT)
  - AGENTS.md containing concise behavioural guidelines for AI coding assistants:
    think before coding and state assumptions, simplicity first (no speculative
    features or abstractions), surgical changes only, and goal-driven execution
    with explicit verification steps.
  - CLAUDE.md containing only the line `@AGENTS.md`.

Initialise a git repository and make the first commit.
```

Verify: `git log` shows one commit; `tree` shows the folders above.

Step 2 — Explore the API and record its rules

Goal: understand how the API behaves before writing code, and write it down. Every fetcher depends on these findings.

```
Before writing any fetcher, explore the MLA Statistics API with curl and record what
you learn in docs/API_NOTES.md. The public documentation is at
https://www.mla.com.au/prices-markets/statistics/api/ and the OpenAPI spec is at
https://app.nlrsreports.mla.com.au/static/openapi.yaml.

Base URL: https://api-mlastatistics.mla.com.au
Endpoints: /report/1 through /report/10
Always send these headers:
  User-Agent: MLA-Data-Fetcher/1.0 (contact: YOUR_EMAIL@example.com)
  Accept: application/json

For each of the ten endpoints, probe it and document:
  1. What the report contains and how often it updates
  2. Accepted query parameters (fromDate, toDate, page, and any filter such as
     category, species, indicatorID, saleyardID, countryID, stateID, year)
  3. Whether a filter parameter accepts multiple values or exactly one per request
  4. The JSON response shape. Expect a top-level "data" array and a
     "total number rows" integer, with 100 rows per page.
  5. The field names in each row (these become the CSV columns)
  6. Any date-range restrictions. Test specifically:
     - a range spanning two calendar years (some endpoints return HTTP 500)
     - toDate equal to today (some endpoints return HTTP 500; yesterday works)
  7. Valid values for each filter (list them fully; e.g. species names,
     indicator IDs with descriptions and units, state IDs, categories)

Also note: there is no authentication, and the API rate-limits aggressive clients
with HTTP 429 / 503, so pace requests at roughly 1.5 seconds between pages.

Write the notes as one section per endpoint with a table of parameters and a
sample row. Do not write any Python yet.
```

Verify: `docs/API_NOTES.md` has a parameter table, field names, and a sample row for every endpoint, and states clearly which endpoints reject cross-year ranges and which reject today's date.

> Answer key for checking the assistant's findings:
>
> - HTTP 500 on cross-year ranges: /report/5, /report/6, /report/10. Split by calendar year.
> - HTTP 500 when toDate is today: /report/4, /report/5, /report/6, /report/10. Default the end date to yesterday.
> - Exactly one filter value per request: /report/2 (stateID, category), /report/4 (category), /report/5 and /report/6 (indicatorID), /report/7 (countryID), /report/10 (species).
> - /report/8 and /report/9 have no filter parameters.
> - /report/2 is queried by `year` and `stateID`, not by dates.

Step 3 — The first fetcher (the template): /report/3

Goal: one complete, robust fetcher that every later report copies.

```
Write fetchers/fetch_report3.py: a standalone CLI that downloads /report/3
(Australian Slaughter and Production, quarterly ABS figures) to CSV.
Use docs/API_NOTES.md for the endpoint details.

Hard requirements:
- Python standard library only (argparse, csv, json, urllib, time, datetime,
  pathlib). No requests, no pandas.
- Module-level constants: BASE_URL, ENDPOINT = "/report/3", PAGE_SIZE = 100,
  REPO_ROOT = Path(__file__).resolve().parents[1],
  DEFAULT_OUTPUT = REPO_ROOT / "data" / "raw" / "report3_slaughter_production.csv",
  VALID_CATEGORIES (the full list from the API notes),
  CSV_FIELDS = ["report_date", "report_type", "category", "location_id",
                "unit_of_measure", "value_amt"].
- build_url(from_date, to_date, category, page) using urllib.parse.urlencode.
- _make_request(url, contact_email) that sends the User-Agent
  "MLA-Data-Fetcher/1.0 (cattle/sheep producer internal use; contact: <email>)"
  and Accept: application/json, with a 30 s timeout, returning parsed JSON.
- Adaptive rate control (AIMD): DELAY_MIN = 1.5, DELAY_MAX = 120, DELAY_STEP = 0.25,
  DELAY_MULT = 2.0. Sleep `delay` seconds between pages. On HTTP 429/502/503/504
  multiply the delay by DELAY_MULT (capped at DELAY_MAX), print a warning, sleep,
  and retry the same page. After every successful page subtract DELAY_STEP
  (floored at DELAY_MIN). Any other HTTP error or URLError is fatal.
- Pagination: request page=1,2,... until a page returns fewer than PAGE_SIZE rows
  or the accumulated rows reach "total number rows".
- The API accepts at most one category per request. If the user passes several
  categories, loop over them and concatenate; if none, make one unfiltered request.
- fetch_all(from_date, to_date, categories, contact_email, progress_callback=None)
  -> list[dict]. When progress_callback is None print progress to stdout; otherwise
  call progress_callback(info) with info["stage"] in
  {"category_start", "page_done", "waiting", "rate_limited", "complete"} and the
  relevant numbers (page, total_pages, rows_done, total_rows, elapsed, eta, delay,
  page_time, wait_time, code). This hook is for a web UI added later.
- CLI progress output per page, e.g.
  "Page 2/8  [████▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓] 25%  200/800 rows  2.1s/page  delay=1.5s
elapsed=6s  ETA 18s"
  plus "Waiting 1.5s before page 3 ..." between pages and a final
  "Fetch complete: 800 rows in 28s  (28.6 rows/s)".
  Implement small helpers _fmt_duration(seconds) and _progress_bar(done, total).
- save_csv(rows, output_path): create parent folders, write CSV_FIELDS in order
  using csv.DictWriter with extrasaction="ignore", print the absolute path.
- argparse options: --from YYYY-MM-DD (default 1 January of last year),
  --to YYYY-MM-DD (default today), --categories NAME [NAME ...],
  --output FILE (default DEFAULT_OUTPUT), --list-categories, --email EMAIL
  (default YOUR_EMAIL@example.com). Validate categories and exit 1 with a helpful
  message on unknown values.
- main(): print a header block (date range, categories, output, contact, start
  time), run the fetch, save, print total runtime, and end with a one-line
  disclaimer pointing to
  https://www.mla.com.au/general/Terms-and-conditions/data-and-information/
- A module docstring describing the report, the response fields and usage examples.

Verify by running:
  python3 fetchers/fetch_report3.py --list-categories
  python3 fetchers/fetch_report3.py --from 2024-01-01 --to 2024-06-30
and confirm the CSV appears under data/raw/ with the expected columns.
```

Verify: both commands succeed; the CSV column order matches CSV_FIELDS; a wider date range shows the progress bar and the wait between pages.

Step 4 — Reports without filters: /report/1, /report/8, /report/9

Goal: copy the template quickly for the three simplest reports.

```
Using fetchers/fetch_report3.py as the template (same structure, same rate
control, same progress output, same progress_callback protocol, same CLI
conventions), create three more fetchers. Copy the file and adapt; do not
introduce a shared module yet, each script must remain standalone.

1. fetchers/fetch_report1.py — /report/1 Australian Red Meat Exports, monthly
   volume by destination country. Data from January 2000. No filter parameters.
   CSV_FIELDS = ["result_date", "country_desc", "meat_type_group_desc", "weight_amt"].
   Default --from is 1 January three years ago; default --to is today.
   DEFAULT_OUTPUT name: report1_red_meat_exports.csv.

2. fetchers/fetch_report8.py — /report/8 US Domestic Cattle Prices (weekly,
   Steiner Consulting). No filter parameters; all indicators come back together.
   CSV_FIELDS = ["indicator_name", "indicator_date", "indicator_units", "indicator_value"].
   DEFAULT_OUTPUT name: report8_us_cattle_prices.csv.

3. fetchers/fetch_report9.py — /report/9 US Imported Meat Prices (weekly,
   Steiner Consulting). Same shape and fields as report 8.
   DEFAULT_OUTPUT name: report9_us_imported_meat_prices.csv.

Remove the category logic entirely from these three (no --categories,
no --list-*). Keep fetch_all(from_date, to_date, contact_email,
progress_callback=None) so the signature is predictable.

Verify each with a three-month range and check the CSV headers.
```

Verify: `--help` works for all three scripts; three CSVs appear in `data/raw/`.

Step 5 — Per-species with year splitting: /report/10

Goal: handle the "one species per request" and "no cross-year ranges" limits. The web app and the merge script both use this report later.

```
Create fetchers/fetch_report10.py for /report/10 NLRS Slaughter (weekly head
count by state × species, published Fridays), following the report 3 template.

Specifics:
- VALID_SPECIES = ["Cattle", "Calves", "Sheep", "Lambs", "Pigs", "Goat", "Deer"].
- Query parameters: fromDate, toDate, page, and optionally species (one value
  per request). CLI option --species NAME [NAME ...] and --list-species.
- The endpoint returns HTTP 500 for a date range that spans two calendar years.
  Split the requested range into per-calendar-year chunks automatically and
  concatenate. Emit a "category_start" progress event with kind="year" for each
  chunk and kind="species" for each species.
- The endpoint returns HTTP 500 when toDate is today. Default --to to yesterday,
  and if the user passes today or later, clamp to yesterday with a printed note.
- CSV_FIELDS = ["result_date", "contributor_state_id", "species_id", "slaughter_count"].
- DEFAULT_OUTPUT name: report10_nlrs_slaughter.csv.
- Export fetch_all(from_date, to_date, species, contact_email, progress_callback=None)
  plus VALID_SPECIES, _fmt_duration and _progress_bar at module level; a web app
  will import them.

Verify with a range that crosses a year boundary, e.g. --from 2024-10-01 --to 2025-03-31,
and confirm rows from both years are present.
```

Verify: the cross-year request succeeds; passing today as `--to` is clamped to yesterday.

Step 6 — Per-indicator splitting: /report/5

Goal: 18 price indicators, one indicatorID per request.

```
Create fetchers/fetch_report5.py for /report/5 NLRS Livestock Indicators
(daily national price indices), following the report 10 pattern (per-year
splitting, yesterday as default end date).

- The endpoint requires exactly one indicatorID per request. Define
  VALID_INDICATORS as a dict {id: (species, description, units)} for IDs 1–18
  using the values recorded in docs/API_NOTES.md, for example
  1: ("Cattle", "Western Young Cattle Indicator", "c/kg cwt"),
  7: ("Sheep", "National Trade Lamb Indicator", "c/kg cwt"),
  16: ("Sheep", "Online Lamb Indicator", "$/head").
- CLI: --indicators ID [ID ...] (default: all 18) and --list-indicators, which
  prints an aligned table of ID, species, units, description.
- Loop indicator × year, emitting "category_start" events with kind="indicator"
  and kind="year".
- CSV_FIELDS = ["calendar_date", "species_id", "indicator_id", "indicator_desc",
  "indicator_units", "head_count", "indicator_value"].
- DEFAULT_OUTPUT name: report5_livestock_indicators.csv.

Verify with --indicators 1 7 over one month.
```

Verify: `--list-indicators` prints 18 rows; fetching two indicators for one month succeeds.

Step 7 — Indicators by saleyard: /report/6

```
Create fetchers/fetch_report6.py for /report/6 NLRS Livestock Indicators by
Saleyard. It is report 5 with one extra dimension: an optional saleyardID
query parameter (CLI --saleyard ID) and an extra "saleyard_id" column.

CSV_FIELDS = ["calendar_date", "species_id", "saleyard_id", "indicator_id",
              "indicator_desc", "indicator_units", "head_count", "indicator_value"].
DEFAULT_OUTPUT name: report6_saleyard_indicators.csv.

Warn in the docstring and README that this is by far the largest dataset
(hundreds of thousands of rows per year) and suggest narrowing by
--indicators or --saleyard. Keep everything else identical to report 5.

Verify with --indicators 1 --from <first day of last month> --to <yesterday>.
```

Verify: runs with and without `--saleyard`; the CSV has the extra saleyard_id column.

Step 8 — Saleyard yardings: /report/4

```
Create fetchers/fetch_report4.py for /report/4 Saleyard Yardings (daily NLRS
head counts by saleyard and category).

- VALID_CATEGORIES = ["Cattle", "Lamb", "Sheep"]. The endpoint requires exactly
  one category per request, so loop over the selected categories (default: all
  three) and concatenate. Optional --saleyard ID adds saleyardID to the query.
- toDate = today returns HTTP 500: default to yesterday and clamp like report 10.
  Cross-year ranges are fine for this endpoint; do not split by year.
- CSV_FIELDS = ["result_date", "category_desc", "state_id", "saleyard_id",
                "tranx_type_id", "head_count"].
- DEFAULT_OUTPUT name: report4_saleyard_yardings.csv.

Verify with --categories Cattle over two weeks.
```

Step 9 — Global cattle prices: /report/7

```
Create fetchers/fetch_report7.py for /report/7 Global Cattle Prices, which
combines NLRS (Australia) and Steiner Consulting (USA) series.

- VALID_COUNTRIES = ["AUS", "USA"]. The endpoint requires exactly one countryID
  per request; loop over the selection (default: both) and concatenate.
  CLI: --countries CODE [CODE ...] and --list-countries.
- Accepts today as toDate and cross-year ranges; no special handling.
- CSV_FIELDS = ["indicator_date", "species_id", "country_code", "indicator_desc",
                "indicator_units", "indicator_value", "currency_code"].
- DEFAULT_OUTPUT name: report7_global_cattle_prices.csv.

Verify with --countries USA over one month.
```

Step 10 — Annual herd and flock figures: /report/2

Goal: the only report keyed by year rather than date, iterated over year × state × category.

```
Create fetchers/fetch_report2.py for /report/2 Australian Herd and Flock
Figures (annual ABS estimates by state, one release per financial year).

- Query parameters are year, stateID, and optionally category; the endpoint
  accepts exactly one stateID and one category per request. No fromDate/toDate.
- VALID_STATES = ["NSW", "SA", "VIC", "QLD", "WA", "TAS"].
- VALID_CATEGORIES = ["Cattle", "Dairy cattle", "Meat cattle", "Sheep and lambs",
  "Sheep and lambs - Lambs under 1 year",
  "Sheep and lambs - Breeding ewes 1 year or over",
  "Sheep and lambs - Marked lambs under 1 year"].
- CLI: --from-year YEAR (default 2015), --to-year YEAR (default current year),
  --states ID [ID ...], --categories NAME [NAME ...], --list-states,
  --list-categories, --output, --email.
- Iterate year × state × category (category omitted when none selected) with the
  same rate control; years with no ABS release simply return zero rows and must
  not be treated as errors. Emit "category_start" events labelled
  "2019 / NSW / Cattle" style.
- CSV_FIELDS = ["financial_year", "region_desc", "subcategory_desc",
                "metric_desc", "estimate_value"].
- DEFAULT_OUTPUT name: report2_herd_flock.csv.

Verify with --from-year 2018 --to-year 2021 --states NSW --categories Cattle.
```

Verify: all ten fetchers exist. Run `for f in fetchers/fetch_report*.py; do python3 $f --help > /dev/null && echo ok $f; done` and confirm every line prints ok.

Step 11 — Weekly merge script

Goal: combine price indicators (report 5) and slaughter counts (report 10) into one wide analysis table.

```
Create analysis/merge_reports.py (requires pandas) that joins
data/raw/report5_livestock_indicators.csv and data/raw/report10_nlrs_slaughter.csv
into a weekly wide table.

Output 1: data/processed/merged_weekly.csv, one row per (week_ending × species_group):
  week_ending          Friday date (YYYY-MM-DD); assign every daily row to the
                       Friday that ends its week
  species_group        Cattle | Lambs | Sheep | Calves | Pigs | Goat | Deer
  national_slaughter   sum of slaughter_count across all states for that week
  slaughter_NSW, slaughter_QLD, slaughter_SA, slaughter_TAS, slaughter_VIC,
  slaughter_WA         per-state head counts
  then for each indicator ID 1-18, two columns named
  "ind_<id> <description> (<units>) price_avg" — weekly mean of indicator_value
  "ind_<id> <description> (<units>) heads_avg" — weekly mean of head_count

Species → indicator mapping (a species row only gets its own indicators; other
indicator columns stay empty):
  Cattle → 1, 2, 3, 4, 5, 12, 13, 14, 15, 17
  Lambs  → 6, 7, 8, 9, 10, 16
  Sheep  → 11, 18
  Calves / Pigs / Goat / Deer → slaughter columns only

Output 2: data/processed/indicator_lookup.csv with columns
  indicator_id, price_col, heads_col, indicator_desc, indicator_units, applies_to
as a legend for the wide table.

CLI: --r5, --r10, --output, --lookup with the defaults above, resolved relative to
the repository root (Path(__file__).resolve().parents[1]) so it works from any
working directory. Create output folders as needed. After saving, print a
coverage summary per species: number of weeks, total slaughter, and how many
price columns are populated.

Verify by fetching reports 5 and 10 for the same recent year and running the merge.
```

Verify: `merged_weekly.csv` has 45 columns (9 base columns plus 18 × 2 indicator columns); Cattle rows have cattle indicator values and empty lamb indicator columns.

Step 12 — Streamlit web app (report 10)

```
Create app.py at the repository root: a local Streamlit web app for /report/10
that reuses fetchers/fetch_report10.py rather than re-implementing anything.

- from fetchers.fetch_report10 import VALID_SPECIES, _fmt_duration, _progress_bar,
fetch_all
- Page config: title "MLA Query Tools — NLRS Slaughter", icon 🥩, centered layout.
- Show a company logo from assets/ (put a placeholder SVG there for now),
  resolved via Path(__file__).resolve().parent / "assets" so it works from any
  working directory. Render it as a base64 data URI in st.markdown.
- Inputs: From / To date pickers (DD/MM/YYYY, default from 1 January last year to
  today), a species multiselect (empty means all), an "Advanced settings"
  expander with the contact email, and a primary "Fetch Data" button.
- On click: validate the range, show st.progress and a live log panel
  (st.empty + st.code). Implement on_progress(info) that maps every
  progress_callback stage to a timestamped log line and updates the bar.
- After the fetch: store the DataFrame in st.session_state; show metrics
  (total rows, number of species, date range), an Altair horizontal bar chart of
  total slaughter by species, the full st.dataframe, and a Download CSV button
  writing only the four CSV columns.
- Keep the last fetch log in a collapsed expander on re-runs.

Also add docs/FEATURES.md summarising the app's features and the CLI output
format in a table. Verify with `streamlit run app.py`.
```

Verify: the browser opens `http://localhost:8501`; fetching one month of data shows progress, the chart, and the download button.

Step 13 — Unified `mla` command line

Goal: one entry point for everything, without rewriting any fetching logic.

```
Create mla.py at the repository root: a thin dispatcher CLI for everything in
the project. It must not duplicate any fetching logic.

Subcommands:
  list        Print a table of the ten reports: ID, short name, update frequency,
              description. Short names: 1 exports, 2 herd, 3 slaughter,
              4 yardings, 5 indicators, 6 saleyard-indicators, 7 global-prices,
              8 us-prices, 9 us-imports, 10 nlrs-slaughter.
  fetch <report> [options...]
              Resolve <report> from "3", "report3", "/report/3" or "slaughter",
              import fetchers.fetch_report<N>, set sys.argv to
              ["mla.py fetch <N>", *options] and call its main(). Every option,
              including --help and --list-*, must reach the fetcher untouched.
              Dispatch this command BEFORE argparse sees the arguments (argparse
              REMAINDER does not reliably pass through options like --help).
  fetch-all [--from] [--to] [--email] [--output-dir DIR] [--only R ...] [--skip R ...]
              Run every selected report in sequence. Translate --from/--to into
              --from-year/--to-year (year part only) for report 2. Only forward
              flags the user actually supplied so each fetcher keeps its own
              defaults. With --output-dir, pass --output DIR/<DEFAULT_OUTPUT.name>.
              Catch SystemExit and exceptions per report so one failure does not
              stop the rest; print a summary table (report, name, seconds, status)
              and exit 1 if anything failed. Handle Ctrl-C gracefully.
  merge [options...]
              Same pass-through mechanism into analysis.merge_reports.main().
  app         Run `python -m streamlit run app.py` via subprocess.

Add `from __future__ import annotations` for Python 3.9 compatibility.

Also add pyproject.toml (setuptools) with:
  name "mla-stats-fetcher", requires-python >=3.9, no core dependencies,
  optional-dependencies analysis = ["pandas>=2.0"], app = ["streamlit>=1.32",
"pandas>=2.0"],
  [project.scripts] mla = "mla:main",
  [tool.setuptools] py-modules = ["mla"], packages = ["fetchers", "analysis"].

Verify:
  python3 mla.py list
  python3 mla.py fetch 3 --help            (shows report 3's own help)
  python3 mla.py fetch herd --list-states
  python3 mla.py merge --help              (shows merge_reports' help)
  python3 mla.py fetch-all --only 1 8 --from 2025-06-01 --to 2025-08-31 --output-dir /tmp/x
  python3 -m venv /tmp/v && /tmp/v/bin/pip install -e . && /tmp/v/bin/mla list
```

Verify: all six commands pass. In particular `merge --help` must show the merge script's own help, not the help of mla.py.

Step 14 — Full English documentation

```
Write the project documentation in English.

1. README.md — expand into a full reference:
   - Intro paragraph and a "Quick Start" section showing pip install -e . and
     the six most common `mla` commands, linking to docs/CLI.md.
   - "Tools Overview" table: mla.py plus one row per fetcher with endpoint,
     data description and update frequency.
   - "Project Structure" tree with one-line comments.
   - "Requirements".
   - One section per report (1–10) with: a short description of the data, a
     Quick Start code block, an Options table (flag, default, description), the
     valid filter values, an "Output Format" sample plus column table, and
     Examples. Mention each endpoint's quirks (year splitting, yesterday cutoff,
     one-filter-per-request).
   - "Merge Reports" section documenting merged_weekly.csv columns and the
     species → indicator mapping.
   - "Web App" section.
   - "API Details" table (base URL, no auth, 100 rows per page, adaptive rate
     limiting) followed by the "Known API limitations" list.
   - "Terms of Use" pointing to MLA's data terms.

2. docs/CLI.md — a complete reference for the `mla` command: installation
   (script vs pip install -e .), a commands-at-a-glance table, one section per
   subcommand with option tables and examples, sample console output, exit
   codes, a "How it works" section explaining the pass-through design and how to
   add a new report, and the terms-of-use note.

3. Update docs/FEATURES.md and docs/API_NOTES.md so all paths match the final
   layout. Use `data/raw/...` and `fetchers/...` paths everywhere.

All documentation must be written in English only.
```

Verify: every command in the README can be copied and run as-is; no old paths remain in any document.

Step 15 — Claude Code skills

Goal: a set of skills so that, in any later conversation, a request like "show me last month's cattle prices" makes the assistant call the right tool and read the right data.

> A Claude Code skill is a `.claude/skills/<name>/SKILL.md` file. It starts with YAML frontmatter (`name` and `description`) followed by instructions for the assistant. The `description` decides when the assistant reaches for the skill, so it must name the trigger situations clearly. Long reference material goes in a `references/` subfolder that SKILL.md links to, keeping the main file short.

```
Create a set of Claude Code skills under .claude/skills/ so that an assistant
working in this repository can access MLA data quickly and correctly. Each skill
is a folder with a SKILL.md that starts with YAML frontmatter:

  ---
  name: <kebab-case-name>
  description: <one or two sentences: what it does AND when to use it, including
               trigger phrases a user might say>
  ---

followed by concise instructions. Keep each SKILL.md under ~150 lines; put long
reference material in a references/ subfolder and link to it.

Create these five skills:

1. mla-api-reference
   Purpose: authoritative reference for the MLA Statistics API as used here.
   SKILL.md: base URL, headers, pagination, rate limiting, the endpoint table
   (ID, short name, data, frequency, filter parameter, one-per-request rules,
   year-split and yesterday-cutoff quirks), and a pointer to one reference file
   per report. Trigger: any question about what MLA data exists, which endpoint
   holds it, what a column means, or valid filter values.
   references/report1.md … report10.md: for each report the query parameters,
   CSV columns with meaning and units, the full list of valid filter values
   (species, categories, states, all 18 indicators with units), a sample row,
   and the default output path. Generate these from the fetcher source files
   so they are exact.

2. mla-fetch
   Purpose: download data with the `mla` CLI. Instructions: always run from the
   repo root; use `mla list`, `mla fetch <report> --list-*` to discover values
   before guessing; how to pick --from/--to (report 2 uses years; reports
   4/5/6/10 need yesterday as the end date); prefer narrow filters on report 6;
   where the CSV lands; how to read the progress output and what a 429 backoff
   means. Include a recipes table mapping common requests to commands, e.g.
   "last 12 months of trade lamb prices" → `mla fetch indicators --indicators 7 --from
...`.
   Trigger: user asks to download, fetch, pull, refresh or update MLA data.

3. mla-refresh
   Purpose: bring every dataset up to date and rebuild the merged table.
   Instructions: run `mla fetch-all --from <1 Jan last year>` (or --only 5 10
   for a quick refresh), then `mla merge`, then report the summary table and
   the date range now covered by each CSV (read the min/max date column with a
```

```
   short pandas one-liner). Note the run takes several minutes and must not be
   parallelised because of API rate limits.
   Trigger: "update the data", "refresh everything", "rebuild merged table".

4. mla-query
   Purpose: answer questions from the data already on disk without hitting the
   API. Instructions: which CSV under data/raw or data/processed answers which
   kind of question; the exact column names and date formats; the species →
   indicator mapping; how to load with pandas, filter by date, resample weekly
   or monthly, and compute year-on-year change; that indicator prices are in
   c/kg (cwt = carcase weight, lwt = live weight) except the "Online" indicators
   in $/head; to check the CSV's max date first and suggest mla-refresh if it is
   stale. Include three worked examples as short pandas snippets (e.g. weekly
   national cattle slaughter vs the National Heavy Steer Indicator from
   merged_weekly.csv; monthly beef exports to Japan from report 1; US imported
   90CL price trend from report 9).
   Trigger: any analytical question about cattle/sheep/lamb prices, slaughter,
   exports, herd numbers, yardings, or the merged table.

5. mla-add-report
   Purpose: extend the project when MLA adds a new endpoint or when a new
   derived table is needed. Instructions: probe the endpoint with curl first and
   record it in docs/API_NOTES.md; copy the closest existing fetcher
   (fetch_report3.py for simple, fetch_report10.py for year-split /
   one-filter-per-request); the module conventions (constants, CSV_FIELDS,
   DEFAULT_OUTPUT under data/raw, fetch_all signature, progress_callback
   stages); add a row to the REPORTS table in mla.py; document it in README.md,
   docs/CLI.md and a new references/reportN.md in mla-api-reference.
   Trigger: "add a new report", "support endpoint /report/11", "new fetcher".

Write every skill in English. After creating the skills, verify each one by
reading it back and checking: the frontmatter parses, every command mentioned
exists (`mla list`, the --list-* flags, file paths), and the references match
the fetchers' constants. Commit them so the whole team gets the skills when
they clone the repository.
```

Verify: `.claude/skills/` contains five folders. In Claude Code, ask "How many cattle were slaughtered in Australia last month?" and the assistant should use mla-query to read `data/raw/report10_nlrs_slaughter.csv`, or suggest a refresh first.

Step 16 — Final checks and commit

```
Do a final quality pass over the whole project:

1. Run every script's --help and every `mla` subcommand's --help; all must exit 0.
2. Run `mla fetch-all --from <1 Jan this year>` once end to end, then `mla merge`,
   and confirm data/raw has ten CSVs and data/processed has two.
3. grep the repository for stale paths (files referenced at the root that now
   live in fetchers/, analysis/, data/, assets/, docs/) and fix any you find.
4. Confirm no personal email is hard-coded anywhere except as an argparse
   default that the README tells users to override.
5. Make sure .gitignore excludes __pycache__, virtualenvs and *.egg-info, and
   that no cache files are tracked.
6. Decide whether the data CSVs should be committed. If they are, say so in the
   README; if not, add data/raw/*.csv and data/processed/*.csv to .gitignore
   and keep the folders with .gitkeep.
7. Confirm all documentation and code comments are in English.
8. Commit with a message that summarises the project state.
```

Verify: `git status` is clean; a fresh clone works end to end by following the README Quick Start.

Appendix: final layout for comparison

```
mla-stats-fetcher/
├── mla.py                    # Unified CLI entry point
├── app.py                    # Streamlit web app (report 10)
├── pyproject.toml            # pip install -e . installs the `mla` command
├── requirements.txt
├── README.md / LICENSE / AGENTS.md / CLAUDE.md / .gitignore
├── fetchers/                 # fetch_report1.py … fetch_report10.py
├── analysis/merge_reports.py
├── data/raw/                 # ten reportN_*.csv files
├── data/processed/           # merged_weekly.csv, indicator_lookup.csv
├── assets/                   # logo
├── docs/                     # API_NOTES.md, CLI.md, FEATURES.md
└── .claude/skills/           # mla-api-reference, mla-fetch, mla-refresh,
                              # mla-query, mla-add-report
```